

System Software

A Level Computer Science

Operating systems

Why a computer needs an OS

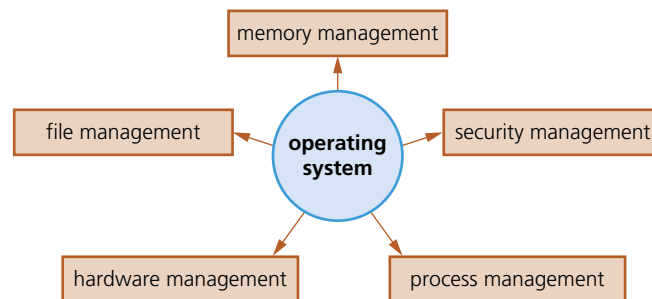
Hardware on its own can only fetch and run instructions —it knows nothing about files, programs, networks or users. The **operating system** 操作系统 (OS) is the **software layer** that:

- **manages the hardware** (**processor** 处理器, memory, I/O, storage) for the running programs.
- **provides services** (file system, network, user accounts) through a clear interface, so programs need not talk to the hardware directly.
- **provides a user interface** (command line, GUI, touch).
- **lets several programs share the hardware safely** —each gets fair CPU time and is kept out of the others' memory.

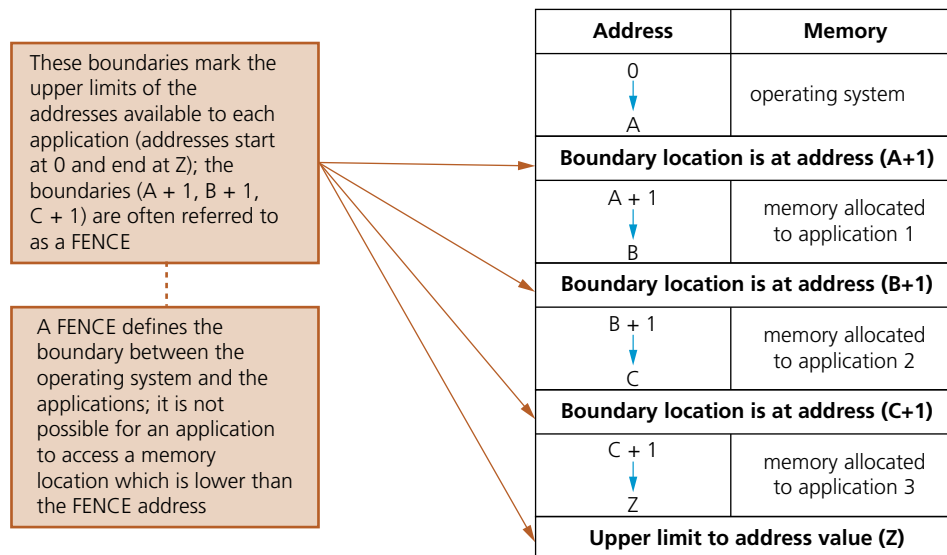
Without an OS, every program would need its own drivers, and only one program could safely run at a time.

Key management tasks

- **process management** 进程管理—load programs, schedule them on the CPU, switch between them, and kill misbehaving ones. (A running program is a **process** 进程.)
- **memory management** 内存管理—give memory to processes, keep them apart, and use **virtual memory** 虚拟内存 / **paging** 分页 so the working set can exceed physical **RAM** 随机存取存储器.
- **file management** —organise files and folders on **secondary storage** 辅助存储器, control permissions, prevent write corruption.
- **device management** —handle I/O through **device drivers** 设备驱动, buffer data, manage interrupts, and give a uniform interface.
- **security management** —accounts, permissions, firewall, encryption.
- **user interface and networking.**



The main jobs the operating system manages



Memory protection keeps each application in its own block of memory

Utility software

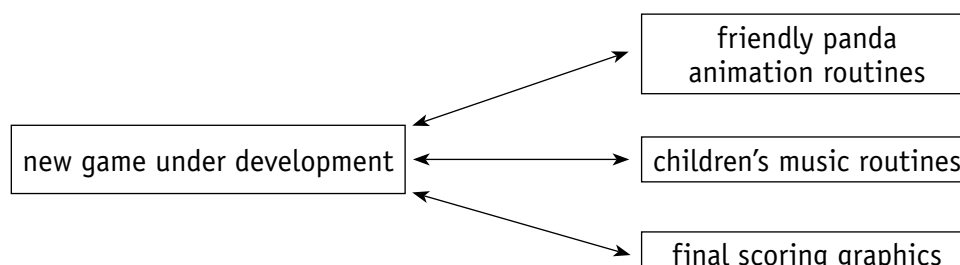
Most OSes include **utility programs** 实用程序 to maintain the system:

- **file / disk management** —copy, move, delete; format; check disk integrity.
- **disk defragmenter** 碎片整理—moves the pieces of fragmented files together on a hard disk to cut seek time (not useful on SSDs).
- **backup** 备份—copies user data elsewhere so it can be recovered.
- **antivirus** 杀毒软件—scans for malware and quarantines threats.
- **firewall** 防火墙—filters network traffic by rules.
- **compression** 压缩 / archiving; **system monitor**; **updates**.

Bundling these with the OS saves the user installing each one.

Program libraries

A **program library** 程序库 is pre-written code (**subroutines** 子程序, classes, modules) that programs reuse instead of writing it themselves —e.g. a maths library, a network library, a graphics library.



A new program reusing ready-made routines from libraries

Benefits: **saves time** (off-the-shelf code), **reliable** (well-tested, widely used), and **standardised** (consistent behaviour).

- a **static library** 静态库 is copied into the executable at compile time (stands alone, but larger and needs rebuilding to update).
- a **dynamic library** 动态库 (DLL, .so) is loaded at run time (smaller executables, shared by many programs, updated once for all).

Language translators

You write source code; the computer runs **machine code** 机器码. A **translator** 翻译器 converts between them.

Assembler

An **assembler** 汇编器 translates **assembly language** 汇编语言 into machine code, one instruction per instruction. Used for low-level code (embedded systems, drivers).

Compiler

A **compiler** 编译器 translates a high-level program into machine code **once, before it runs**.

- it reports **all** errors at compile time; once clean, it produces a stand-alone **executable** 可执行文件 that runs **without the compiler installed** and can be run many times.
- generally **faster at run time** (no translation while running), but tied to one CPU/OS — recompile for each platform.

Interpreter

An **interpreter** 解释器 translates and runs a high-level program **one line at a time**, producing no executable.

- it reports an error **when it reaches that line**, then stops; you can fix it and continue — good for development.
- the **interpreter must be installed** to run the program; generally **slower** (each run re-translates), but easy to **port** across platforms.

Choosing between them

Use a **compiler** for maximum run-time speed, to distribute to users without dev tools, or when the program runs many times. Use an **interpreter** for fast development cycles, cross-platform scripts, small or run-once programs, and teaching beginners.

Hybrid: Java

Java is compiled into **bytecode** 字节码 (a platform-independent intermediate form), which a **virtual machine** 虚拟机 (the JVM) then interprets —or uses **just-in-time compilation** 即时编译 to turn hot parts into native code. So errors are caught early, the bytecode runs anywhere with a JVM ("write once, run anywhere"), and long-running programs reach near-native speed. C# and Python use similar designs.

Integrated Development Environment (IDE)

An **integrated development environment** 集成开发环境 (IDE) brings the tools to write, test and debug code into one application:

- **source code editor** with **syntax highlighting** 语法高亮 (keywords, strings, comments in different colours), auto-indent and bracket matching.
- **auto-complete** 自动补全—suggests names and shows function parameters as you type.
- **translator integration** —compile/run with one keystroke; errors shown inline.
- **debugger** 调试器—set **breakpoints** 断点 to pause, **step through** line by line, and **inspect variables**.
- **version control** 版本控制 integration (git), **project management**, a help system, **refactoring** 重构 tools (safe renaming), and **unit test** 单元测试 integration.

An IDE speeds development by putting writing → running → debugging → fixing behind one interface. Common IDEs: Visual Studio, PyCharm, Eclipse, VS Code.